RAPPORT DE GROUPE : GYM MANAGER PRO

Module : Programmation Python 2

Année Universitaire : 2025-2026

Équipe de Développement :

- **Youness Habzhour :** Architecte Back-End & Tech Lead
- **Noureddine Henni :** Responsable Persistance & Stockage
- **Mohamed Amine Hdidouch :** Développeur Front-End & Interface

---

1. Sujet du Projet et Format des Données

Le Sujet : Notre projet, "Gym Manager Pro", est une application de bureau Python destinée à gérer les abonnements des membres d'une salle de sport. Elle permet d'effectuer des opérations CRUD complètes (Ajouter, Voir, Rechercher, Supprimer) , de générer des alertes pour les abonnements expirés , et d'assurer la sauvegarde automatique des actions de l'utilisateur.

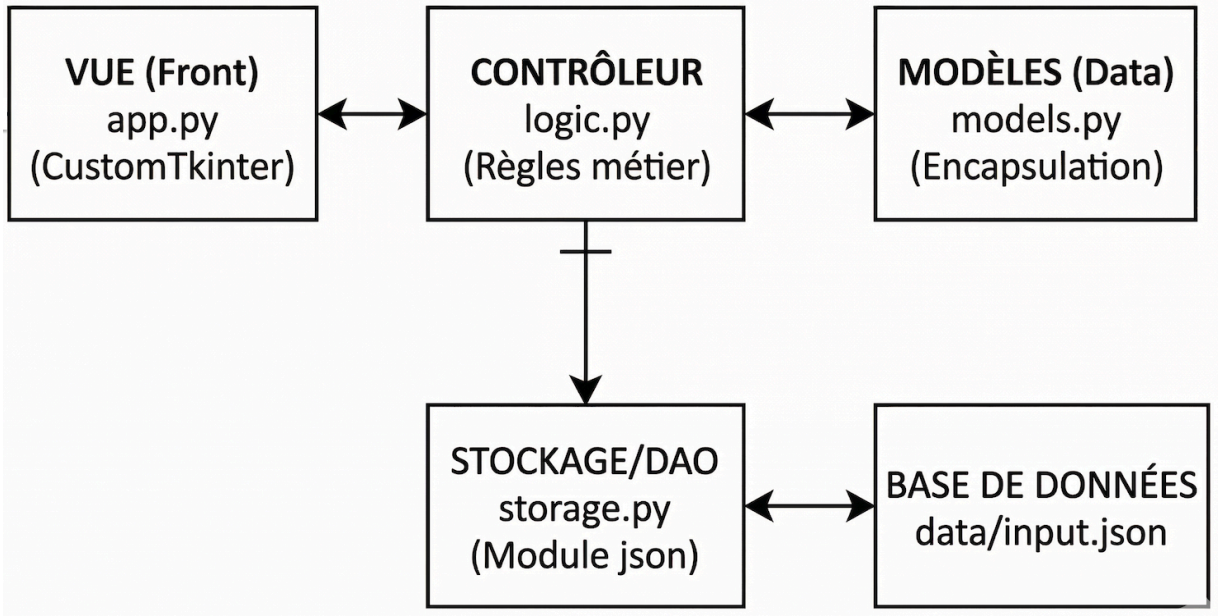
Le Format des Données : Pour répondre aux exigences de persistance, nous avons opté pour le format **JSON**. Ce choix technique nous permet de manipuler facilement des structures complexes. Le fichier principal, situé dans le répertoire data/input.json , stocke les informations sous la forme d'une liste de dictionnaires. Chaque objet membre possède les clés suivantes:

- "nom" : Le nom complet de l'abonné.
- "date_debut" : La date d'inscription (format JJ-MM-AAAA).
- "date_fin" : La date d'expiration de l'abonnement.
- "statut" : L'état actuel ("Actif", "Expiré", "Suspendu").

---

2. Schéma d'Architecture (Modèle MVC)

Pour garantir un code robuste et maintenable, notre équipe a strictement appliqué l'architecture MoΩdèle-Vue-Contrôleur (MVC).



Rôles des composants :

- **L'Interface (app.py) :** Gère exclusivement l'affichage et les clics utilisateurs.
- **Le Contrôleur (logic.py) :** Fait le pont entre l'interface et les données, gère les calculs mathématiques et la génération de rapports.
- **Les Modèles (models.py) :** Sécurise les entités métiers avec des validateurs (getters/setters).
- **Le Stockage (storage.py) :** Gère de manière isolée la lecture et l'écriture sur le disque dur.

---

3. Stratégie de Gestion des Erreurs

Afin de répondre à l'exigence de robustesse, nous avons mis en place une stratégie de défense "en couches" (multi-niveaux) pour éviter tout arrêt brutal de l'application.

A. Au niveau du Stockage (storage.py) : Le module de sauvegarde intègre des blocs try/except spécifiques.

- **FileNotFoundError :** Si le fichier ou le dossier data/ est absent, le système l'intercepte silencieusement et démarre avec une liste vide. Le fichier est recréé à la première sauvegarde.
- **JSONDecodeError :** Si le fichier est corrompu par une modification externe, le système gère l'erreur pour ne pas faire crasher l'application.

B. Au niveau de la Logique et des Modèles (logic.py & models.py) : Nous avons appliqué le principe du "Fail Fast" (Échouer rapidement). Si une donnée est invalide (par exemple, une date de fin antérieure à la date de début, ou un mauvais format de date), les *setters* ou le contrôleur soulèvent intentionnellement une exception ValueError. Cela bloque la création d'objets corrompus à la racine.

C. Au niveau de l'Interface (app.py) : L'interface graphique est conçue pour capturer les erreurs ValueError remontées par la logique. Au lieu d'afficher une erreur technique dans la console, l'application utilise des boîtes de dialogue (messagebox.showerror()) pour informer l'utilisateur de son erreur de saisie de manière conviviale.